BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA

FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

SPECIALIZATION COMPUTER SCIENCE

DIPLOMA THESIS

An approach for patient-doctor relationship management

Supervisor

Lect. Dr. Radu-Lucian Lupșa

Author

Cosmin Holcan

2022

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA

FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ

SPECIALIZAREA INFORMATICĂ

LUCRARE DE LICENȚĂ

O modalitate de gestiune a relației pacient-doctor

Conducător științific

Lect. Dr. Radu-Lucian Lupșa

Absolvent

Cosmin Holcan

2022

ABSTRACT

The purpose of this thesis is to investigate the patient-doctor relationship in the context of technological progress in the last decades and how this relationship can be facilitated through a web application. In the first part, we have identified the reasons for making such an application, like the improvements in patient safety, better coordination for both participants in this interaction and the fact that doctors can make a better analysis of the evolution of the disease and patient life values. In the second part there is described the implementation process. We established the main functionalities that would improve the patient-doctor relationship in comparison with an exclusively physical interaction, these being real-time communication, a special section to write the observations of doctor and patient on the evolution of the disease, the possibility of making an appointment and prescribing recipes.

Contents

1 Introduction

The problem that this thesis brings up is a well-known one globally, how the patient-doctor relationship could be implemented in a software system in order to facilitate the interaction process between these two parts. We have all noticed the technological progress that has been made lately and we can state that the time when the only way a patient could ask for a doctor's advice was by visiting his office is a thing of the past. In the last decades, web applications have developed continuously and become a great option to digitize tasks that in the past required various resources. Thus, we can use this progress in web applications to develop a solution that meets the main requirements of patient-doctor interaction.

My inspiration behind this work comes from the fact that, at this time, an important problem among the population is given by misinformation about different kinds of health problems, people falling into the trap of looking for answers on their own and not asking for advice from an expert. At the same time, another argument is represented by the fact that digital tracking of the progress of treatment could improve the current situation and be easier to follow. These being the reasons behind my thesis idea, I intend to develop a web application that can be easily used by patients and doctors, without special training in advance. This application should come in support of an ordinary person who needs the opinion of a doctor or wants to make an appointment for a consultation, while also helping the doctor in organizing his or her work.

1.1 Objectives

- Describe the main types of the patient-doctor relationships and how do they affect the development of the application
- Why and how to develop a web application in order to facilitate this interaction?

- Present how web applications have developed, what is their structure and how a real-time communication can be achieved
- Develop a web application to facilitate the patient-doctor relationship
- Present the technologies used in the process of development

1.2 Thesis outline

The content of this thesis is structured in four parts:

The first part is represented by Chapter 2 – Theoretical Background and presents at the beginning the evolution of the relationship between patient and doctor through the history and classification of this interaction. Furthermore, there is described the link between the patient-doctor relationship and web applications: what are the advantages, possible problems to take into consideration of this approach, and how such a system should be developed.

The next part, Chapter 3 – Web applications, is dedicated to web applications, focusing on how they have improved over the years and what is their structure; also there is a section dedicated to web sockets.

The third part, Chapter 4 – Application Description, shows the process of application's development. Firstly, there are presented the features of the application, how the pages of the application look and behave. Furthermore, there is described the process of implementation, offering explanations for the choices that were made. The last section is dedicated to the used technologies during the development process.

The last part, Chapter 5 – Conclusions, as the name suggests, presents the obtained results of this thesis and how the developed application can be improved.

2 Theoretical Background

2.1 Patient-Doctor Relationship

2.1.1 Brief History

From ancient times, people have sought the help of doctors to solve medical problems, but the relationship between the two entities has not always been similar to that of today. Prior to the last two decades, the relationship was predominantly between a patient seeking help and a doctor whose decisions were silently complied with by the patient [8]. We can consider as reference points in the evolution of this relationship, the periods:

a) Ancient Egypt (approximately 4000 to 1000 B.C)
b) Greek enlightenment (approximately 600 to 100 B.C)
c) Medieval Europe and the inquisition (approximately 1200 to 1600 A.D)
d) The French Revolution (late 18th century)
e) Patient-doctor relationship 1700-to present day

The period of ancient Egypt shows the position of the doctor through that of the healer, who explains the unknown through magic and mysticism, but who still uses rational methods of healing diseases or wounds. The patient did not necessarily have a say in the practices used to heal him/her, the doctor made all the decisions. The Greeks developed a system of medicine based on an empirico-rational approach, such that they relied ever more on naturalistic observation, enhanced by practical trial and error experience, abandoning magical and religious justifications of human bodily dysfunction. Thus guidance-co-operation and to a lesser degree mutual-participation were the distinguishing patterns of the doctor-patient relationship [8].

During medieval Europe, there was a regression both in terms of healing techniques and the actual interaction between doctor and patient. The practice of magic has taken shape again and

the status of the patient is that of a man without opinions, who is just waiting to be saved. The French Revolution and its aftermath seem to prove that history is repeating itself. In this way, a correlation can be noticed between the scientific progression and the guidance-co-operation of doctors and patients.

Since the 1700s, thanks to impressive medical advances, the development of technology, and society in general, the patient-doctor relationship has evolved through multiple intermediate phases (for an example, treating the sick one as a person, and not as a human experiment) to the one we know today, of patient-oriented. This assumes that the doctor is aware of the patient's personality, informs him/her of the possible treatments and the two jointly decided on which one to use. In 1955, Michael Balint firstly stated that the communication between doctor and patient is as important as the medicine: by far the most frequently used drug in general practice was *the doctor himself*. It was not only the medicine in the bottle, or the pills in the box, that mattered, but the way the doctor gave them to his patient-in fact the whole atmosphere in which the drug was given and taken [3].

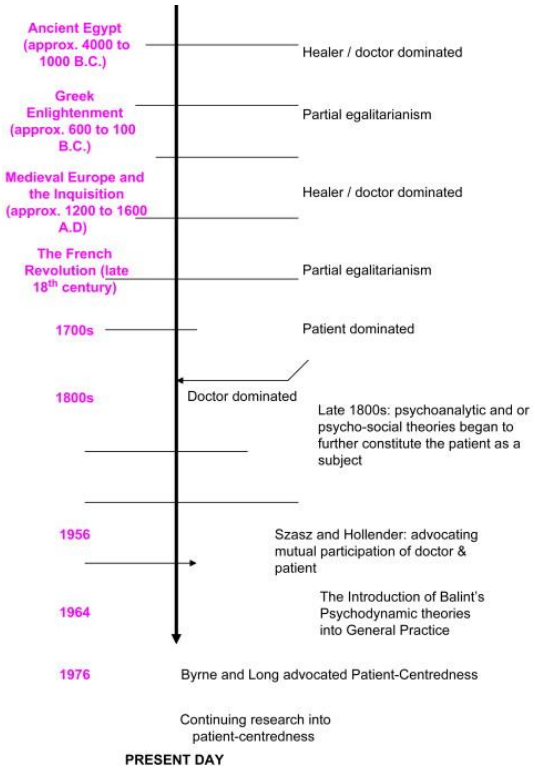

Figure 2-1 Evolution of patient doctor relationship [8]

2.1.2 Different Types of Patient-Doctor Relationship

In order to understand the importance of technology in the patient-doctor relationship nowadays, we need to have a clearer view of this domain and that implies knowing the main types of this interaction, but also which ones have the best results and what are the trends in this field.

Doctors Ezekiel and Linda Emanuel proposed in 1992 a classification of the physician-patient relationship based on the following criteria: (1) the goals of the physician-patient interaction, (2) the physician's obligations, (3) the role of patient values, and (4) the conception of patient autonomy [6]. This classification contains four models: the paternalistic, the informative, the interpretive, and the deliberative, but as the authors themselves suggest, there could be also more models.

The paternalistic model involves the doctor establishing what the best option for the patient's health is and trying, with the help of the acquired medical knowledge, to apply the necessary practices for healing. The patient receives the information regarding the practices that suits best his or her needs and it is possible to disagree initially with the doctor's approach, but later he or she will be satisfied with the taken decisions. This model has as a counter-argument the fact that doctor and patient don't share the same view in every situation; sometimes the patient can have different interests than its health (for example, a future grandmother chose a safe but not so effective treatment before the birth of her first grandchild, which will take place in a few months, instead of a more effective practice but still not sufficiently tested). Of course, there are special cases in which this type of interaction is justified, like in critical situations where time is a crucial factor in making a decision.

In the informative model, the doctor presents to the patient all the medical information that is required for this to decide on what treatment should be used. In this way, a doctor is just a tool; his or her impressions about the case don't matter and have as only purpose to provide facts for the patient, like the disease state, the nature of possible diagnostic and therapeutic interventions, the nature and probability of risks and benefits associated with the interventions [6]. Important to notice that this model is more a theoretical one because in real-life situations there are a few

chances that a person only wants to know the scientific information and doesn't need at all the subjective impression of the doctor.

The interpretive model is similar to the informative one to an extent because it implies that the doctor comes to the patient with all required technical information, but in this case, also helps the person in need to understand what are his or her real intentions. The patient can be unsure regarding what he or she really wants or needs and the doctor has the mission to clarify what are the goals of the respective person and how they could be achieved. This model encounters difficulties due to the lack of proper training of the doctor in interpreting the patient's wishes, but also the time-limited interaction between the two.

In the last model, the deliberative one, the doctor aims to determine the best medical solutions for the patient's problem, but also to convince through moral persuasion the sick person to choose the required interventions to improve his or her health. If the patient does not agree with the doctor's recommendations, then they deliberate about other alternative healing approaches according to the patient's interests, after all being about his or her own life. Of the four presented models, the deliberative model seems to be the most successful option for multiple reasons. One of them is the fact that it respects the autonomy of the patient; even if the doctor has the goal to promote the practices with the best results from the point of view of health-related values, the patient is still free to choose what is better for his or her interests. Also, the doctor is seen as a caring person, who wants to heal the sick one and not just presenting multiple facts about the disease state and risk degree of possible interventions. This role approaches the general perception of society toward the doctor.

Comparing the Four Models

	Informative	Interpretive	Deliberative	Paternalistic
Patient values	Defined, fixed, and known to the patient	Inchoate and conflicting, requiring elucidation	Open to development and revision through moral discussion	Objective and shared by physician and patient
Physician's obligation	Providing relevant factual information and implementing patient's selected intervention	Elucidating and interpreting relevant patient values as well as informing the patient and implementing the patient's selected intervention	Articulating and persuading the patient of the most admirable values as well as informing the patient and implementing the patient's selected intervention	Promoting the patient's well-being independent of the patient's current preferences
Conception of patient's autonomy	Choice of, and control over, medical care	Self-understanding relevant to medical care	Moral self-development relevant to medical care	Assenting to objective values
Conception of physician's role	Competent technical expert	Counselor or adviser	Friend or teacher	Guardian

Figure 2-2 Comparison between the Four Models [6]

Important to notice that the classification made by Ezekiel and Linda Emanuel is based on two dimensions, the autonomy and the values of the patient, and these two dimensions are considered to be strongly connected, which is not really the case in all real-life scenarios as Agarwal and Murinson stated, observing that the medical knowledge of the patient can represent the third dimension. Also, they made an axis interpretation of the four models: a shift from completely unformed to fully formed values - and a corresponding shift from low patient autonomy to high patient autonomy - occurs as one progresses from the paternalistic approach to an informative one. Visually, this can be represented as a single axis in which the extent of values formation and patient autonomy are mutually varying [1].

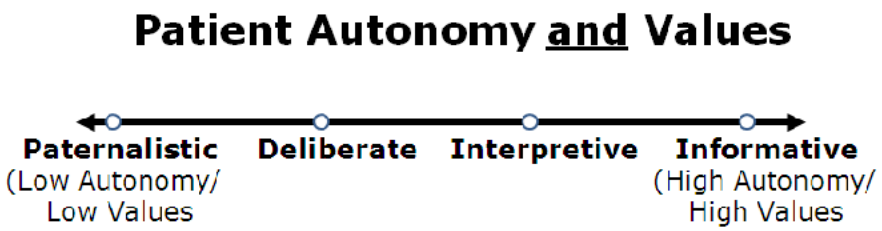

Figure 2-3 Axis interpretation of Emanuel and Emanuel Models [1]

The trend in the patient-doctor relationship domain has undergone major changes throughout history, but as it is pointed out by many specialists, over the past few decades, there has been a shift from a paternalistic approach to patients towards an emphasis on patient autonomy [4]. Most certainly no model is perfect; on the one hand, each model has objections, on the other hand, there are situations for each one in which its use is justified. If we were to choose one as the best option in a digital age in which the right to an opinion is extremely important, then the deliberative model comes closest to meeting all the criteria of specialists. Such a model, which is based on open dialogue, joint deliberation, and mutual understanding, would be the balanced alternative between the two extremes represented by the paternalism and the informative model, as it overcomes the imbalance of decision making power between patient and physician [4].

2.2 Patient-Doctor Relationship in a Software System

2.2.1 Why and how to implement a software system for patient-doctor relationship?

Digitization is a process that we have all been following for decades. The medical field is no exception; this case raises numerous questions related to the development of a software system that would improve the patient-doctor relationship. The first question would be why to implement a software system that tries to solve this problem, or in other words, if it is really necessary for such an implementation.

A first argument is given by the fact that a software system to manage the patient-doctor interaction improves patient safety. It is well-known that some patients do not remember which pills to take, how many are needed, or at what intervals they should take their medication. This problem can be easily solved if the doctor introduces the patient's prescription with the help of an application. In this way, the patient can verify the exact information at any time that he or she wants.

Another reason is that it would be much easier for patients to make an appointment, which would lead to better coordination for everyone. For a patient to be consulted in a doctor's office, excluding emergency cases, one of the following scenarios would be possible: to go to the office without an appointment, contact the doctor by phone, or make an appointment through an application. The first option is clearly the wrong one because the doctor's schedule for that day may be full. The second option has the disadvantage that the patient has certain time intervals when he or she is available, but not knowing the doctor's schedule the patient may be forced to change his plans in an irritating way. Also, this choice requires time from the doctor to make the appointments, time that could be used more efficiently. Therefore, the third possibility is really the best, automating the process.

A third argument is the fact that through a software system it would be much easier for doctors to make a general analysis of the evolution of a patient's disease and what are his or her values. In this sense, a notes feature can be implemented for patients to write what they feel about their situation, what they noticed over time and doctors can access these notes, but also write their observations. A parallel between these two hypostases can help doctors better understand what patients really want and how to manage consultations with them. This is an important aspect as we already saw that the approach used today in patient-doctor interaction is a deliberative one. If the doctor has a clearer view of the patient's inner thoughts, it would be more facile to select a treatment appropriate to the problem that he or she has and to present the situation to the patient.

In addition, a real-time chat would help in establishing a stronger relationship between patients and doctors. It is important for the doctor to win the patient's trust so that he or she can follow the prescribed treatment, the moral factor greatly influencing the entire healing process. At the same time, a real-time chat can be used in other patient-doctor models like the informative one. There can be patients with some medical knowledge that just need information in order to make a decision, so the doctor can answer his or her questions through direct messages.

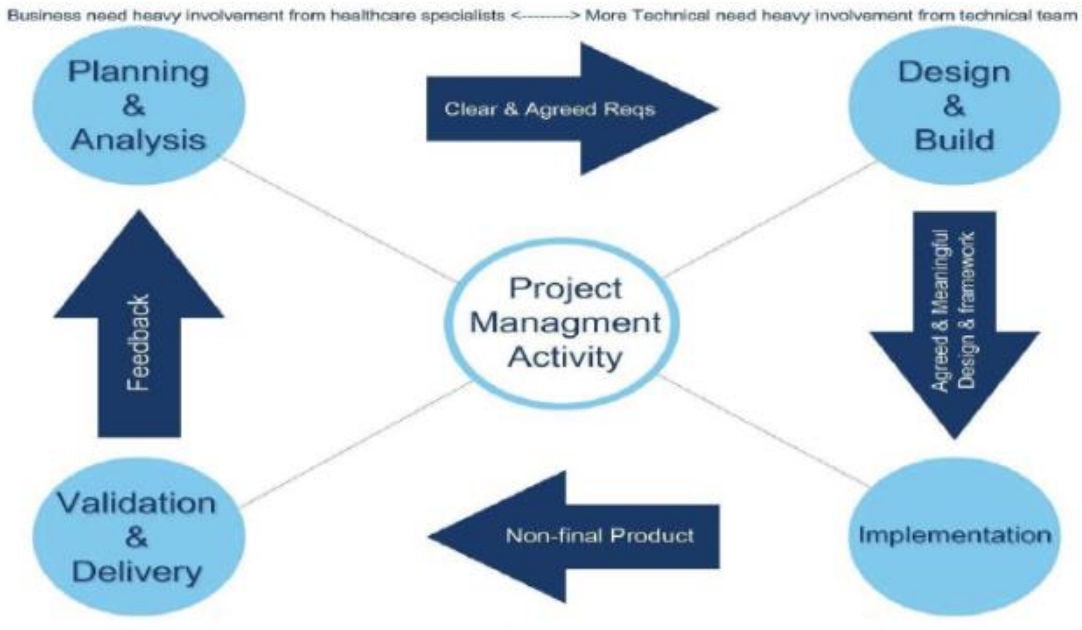

Figure 2-4 Software Engineering Methodology for Healthcare Applications [2]

After seeing the arguments for developing a software system for managing the patient-doctor relationship, we will turn our attention to how this process should be done. In article [2], there is proposed a new software engineering methodology for healthcare applications SEMHTA. This methodology considers the healthcare background of the end-user and the impact on the development activities. SEMHTA is built on four main phases: Planning and Analysis, Design and Build, Implementation, Validation and delivery [2]. Each phase requires multiple activities and all phases are strongly related to others. SEMHTA framework is presented in Figure 2-4.

The Planning and Analysis Phase is arguably the most challenging one when developing a healthcare application. Initially, there must be created plans which have the aim to guide the development and healthcare team throughout the entire process. Also, there should be built communication channels between the healthcare specialists and software engineers. During the analysis phase, the main activity is to gather hospital's requirements and environmental considerations [2]. In SEMHTA methodology, this can be achieved through workshops where different categories will participate like physicians, developers, or even possible patients.

The Design and Build Phase aims to create the technical plan to implement the requirements. This phase requires the plans from the previous stage based on which the design document will be created. In this document, different technical aspects like the used programming languages, machines, and the architecture will be specified. The healthcare specialists should have an important contribution to this document. In the Implementation Phase, the application is developed based on the components identified during the design period. There are followed some concerns like reliability, performance and security.

The SEMHTA methodology suggests that the Validation and Delivery Phase should be executed by a joint team and not by another team as is the case in many other software engineering methods. This joint team includes the developers of the application but also healthcare specialists and end-users. Assigning the validation phase to a total different team seems to lead to many errors.

The same research paper [2] presents a comparison between some software engineering methodologies, based on the healthcare system issues such as reliability, performance, and availability. The conclusions of the study show that the SEMHTA is the best approach to

developing a healthcare application compared to Waterfall, Incremental and Agile methodologies. At the same time, it is stated the obtained results are valid only for the development of healthcare applications.

2.2.2 Problems with a software system for managing the patient-doctor interaction

We must specify from the very beginning that a complete transition of the patient-doctor relationship to the online environment is not yet possible. Face-to-face communication is extremely important in establishing any kind of interpersonal relationship, but in this field, it is all the more necessary as a consultation often involves a physical evaluation of the patient. At the same time, the creation of an application to manage this interaction between doctors and patients is clearly justified, as we could see in the previous subsection.

As in any application, a major issue when developing a software system for managing the patient-doctor relationship is performance. This aspect can be seen from multiple perspectives, one being the performance related to the data accuracy and the data availability. Another problem is represented by the performance from the patient's view who wants privacy and a decrease in waiting time. Last but not least, there are issues from the perspective of the medical staff like the performance of the communication between doctors and patients and the integrity of the system.

Reliability is also a problem, but it is not entirely specific to the type of application we want to make, being more of a problem related to the software development itself. On the other hand, availability represents a critical problem when developing software for managing the patient-doctor relationship. Availability is defined as the percentage of time when the system is up and running [5]. It represents such an important issue for an application of this type because of the nature of its purpose: there can be cases when patients really need help and there is no time to waste.

An issue that has been debated countless times regarding the development of a medical application is its security. It is normal to be an important issue as the personal data of the patient are present and must be protected. In the research paper [7], there are presented some security problems and how they can be solved. One of them is the unauthorized access to patient' electronic records. When roles and privileges are not properly defined among users of the e-health database, it means anyone can access information and carry out any type of actions even if it's a malicious action; this is a security threat to the e-health system [7]. A solution for this issue is represented by using the Nu.Sa (Nuvola Sanitaria) framework, which is based on collecting the data like electronic health records (EHR) from patients and combining them into a cloud system architecture. This architecture provides privacy and security through data encryption algorithms [7].

Another security concern is represented by tracking users' activities. Let's imagine the case when someone can see a patient's oxygen saturation and heart rate data. Having this information, it is an easy task to deduce how regularly that patient is doing physical activity. With this kind of information, access to insurance benefits could be restricted by insurance companies for users with delicate life styles by taking undue advantage of the information and by so doing, compromising the user's right of privacy [7]. A solution to this problem is that the database administrators should establish the exact privileges rules in the application with the aim to restrict access to sensitive information.

2.2.3 Access to information rights

An important problem in a software system on the subject of the patient-doctor relationship is represented by who has the right to access medical information regarding a certain patient. Of course, the first observation we can make is that a patient should only have access to his or her data, not being allowed to obtain data from another individual. At the same time, a doctor should have access to data found by him or her (like prescriptions), but the real dilemma is whether another doctor should have access to information from a patient he or she has not consulted.

To find an answer to this question, we should first see who has the ownership rights on this kind of information. The situation is not so clear in any country, this being in itself a legal matter. As it is presented in the article [13], in the USA, the patients are not the ones to have the ownership rights to their data, but this right is assigned to the medical providers. What must be said is that these rights are limited and medical providers must respect ethical and legal principles in the use of data regarding patient privacy. In the same article, there is conducted a series of interviews with some specialists in the domain. One presented idea is that the concept of ownership may not be beneficial for our problem as it can restrict the process of making the best decisions for a medical problem. The results show that many of the experts they interviewed did not think that individuals should be able to assert broad ownership claims over the health, medical, and genomic data shared with a MIC (Medical Information Commons) [13]. At the same time, it is also stated that the patients keep some rights that should include, at a minimum, transparent communication, individual access to that information, and perhaps even some degree of control over its use [13].

Bearing in mind that patients do not necessarily own their medical records, we strongly believe that other doctors should have access to their data. At the same time, they should not have access to all of a patient's data unless he or she grants them this privilege. It would be probably a good idea for each patient to have a specific general medical history that is used in emergency cases and is accessible to any doctor. Otherwise, regarding the other non-essential data, if they were not described by the current doctor and the patient avoids making them public with him or her, being aware this action can worsen the treatment process, they should not be shared. In this case, we take into account the deliberative model of the patient-doctor relationship, the best as we presented earlier, which suggests that in the end, the patient is in control of his or her own life.

3 Web applications

3.1 Brief history

Web applications are part of our daily life thanks to their ease of use by a regular user, who does not need programming knowledge, but also to their applicability in various fields, starting from e-commerce, social networks and ending with the domain that attracts our attention, the medical one. Web applications started to develop in the 90s, being initially only static HTML pages, after that being possible to add media files, but they were still static. An important element in making dynamic pages as we know them today was the invention of JavaScript in 1995, allowing animations to be displayed. The move from static web pages to dynamic ones was materialized in 2005 with the creation of Ajax (Asynchronous JavaScript And XML) technology. In fact, the problem with the traditional web applications was that they used a synchronous request-response protocol which led to a delay between pages. On the contrary, Ajax applications are based on asynchronous requests, which leave the user interface active and responsive [12]. Since the point where Ajax technology was introduced, the domain of web applications has developed continuously, both on the backend and frontend sides. Modern web applications are considered to provide multiple benefits like flexibility, scalability, interactivity and also they should be cloud-based and compatible across different platforms.

3.2 Structure

Web applications have a specific architecture, consisting of several logical and physical divisions named tiers, the most common architecture being the three-tier one, having the following tiers: the presentation tier, the application tier and the data tier. The main advantage of the three-tier architecture is the fact that each tier runs on a separate infrastructure, this being extremely important in the development process. In this way, a separate team can be assigned to each tier

and all three teams can work simultaneously. Also, any required update on a specific tier can be implemented easily because the changes do not impact the other tiers. Not to forget, the user can't access directly the database and this guarantees a certain degree of security, but it can still present problems. Related to problems, the three-tier architecture is not perfect and the main disadvantages are its complexity to build (in comparison with a two-tier architecture, in which the connection between the client-side and the database is made directly) and the fact that it is more costly in terms of time (an example is that even a small update has to be tested in multiple places).

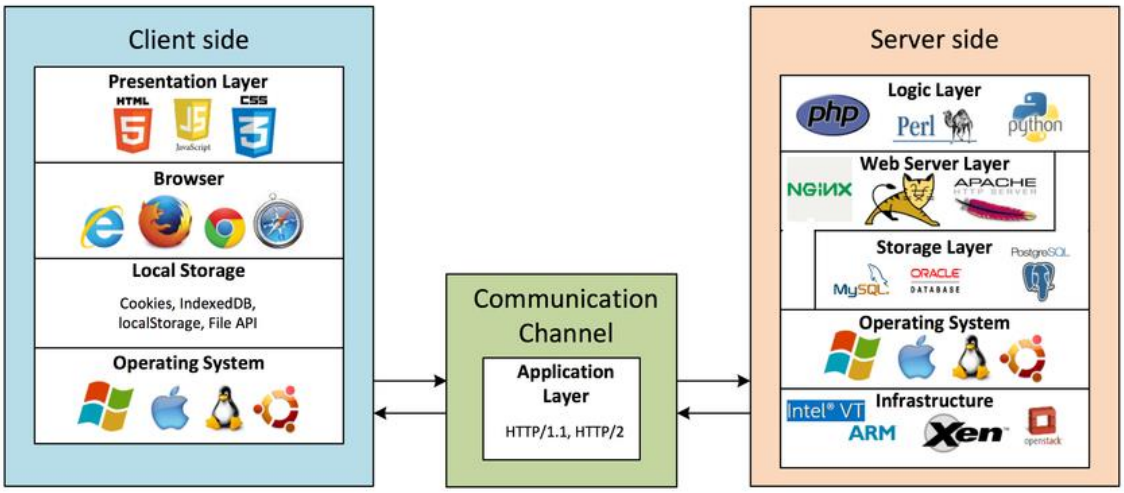

Figure 3-1 Structure of a three-tier web application [16]

As it can be seen in figure 3-1, there are two sides of a distributed web application, the client-side and the serve-side, the link between them being made by the application tier. The client-side deals with the interaction between the user and the computer, being responsible for converting the HTTP responses from the server into actionable graphical interface for the user [16]. It consists of multiple components:

- The presentation layer - deals with the logic on the front-end side, it represents the code of the top tier of the architecture

- The browser – takes the presentation layer code and interprets it in order to be displayed and to behave accordingly to its purpose
- The local storage – is used by the presentation layer to store data. Examples include cookies, localStorage, IndexedDB and File APIs [16]
- The operating system – the system on which the browser is running

The client-side communicates with the application tier through HTTP protocol. The client makes requests with the purpose to access servers' resources and receives a response for each one of them. A well-defined HTTP request contains 3 elements: a request line, a variable number of HTTP headers and optionally a message body. The request line consists of its method, which should identify the type of request by the purpose in which the resources from the server will be used, and its path, but can contain other elements like a query string, which provides information that can be used by application tier in different ways (for example, to extract specific information from the database). HTTP headers have the purpose to inform the recipient about the message, the sender and the way in which the communication should happen.

The application tier represents the place where the data that comes from the presentation tier or the database tier is processed, being the center of the architecture. It can be seen as an intermediary tier between the other two. The communication between the application tier and the database tier is made also usually through HTTP protocol, using API calls.

The server-side, as it can be seen in figure 2-4, consists of the following components:

- The logic layer – implements the business logic of the data tier in a high-level programming language like Java or C#
- The web server layer – receives HTTP requests from clients, parses them, and passes the request to the appropriate server-side program. Examples of web servers include Apache web server, Windows IIS, and Nginx [16]
- The data storage layer – in fact, this layer is the real database, the place where the whole information of the application is stored
- The operating system – is necessary for the other layers to run on

Another type of architecture is the two-tier one, based on the client-server model, in which the logic of the application tier is included in the data tier. In this way, the communication between the client and the server is directly made, increasing the execution speed of the application. The problems with this architecture are represented by its lack of scalability in the case of a high number of users and its weak security.

3.3 Web Sockets

The web application that we are trying to build in this paper involves direct communication through messages between doctors and patients. Basically, this feature is distinctive of a real-time application and in order to develop an application as efficiently as possible, technologies specific to this type of application are needed. That being said, web sockets have caught our attention to solve this problem.

Web Socket is defined as a technology that enables web pages to use the Web Socket protocol for full-duplex communication with a remote host [11]. The Web Socket protocol is an application-level protocol built on top of the TCP. It enables bidirectional data transport in Web sessions and is considered as a viable alternative to HTTP polling techniques, which are implemented as tradeoffs between functionality, efficiency and reliability [15].

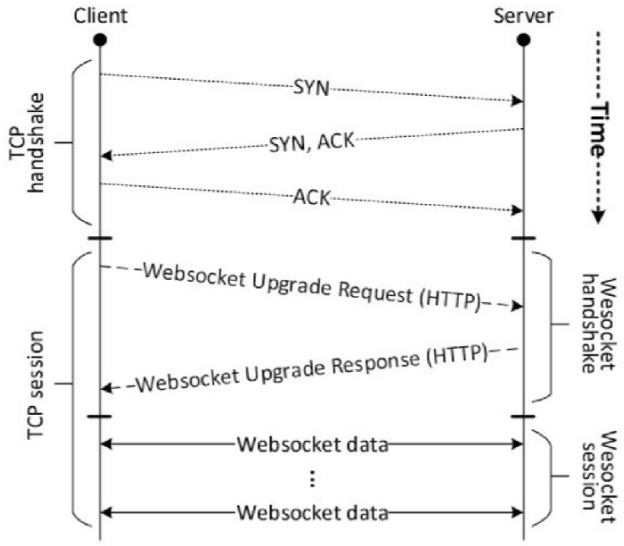

Figure 3-2 Web socket over TCP sequence diagram [15]

In figure 3-2, it is presented a sequence diagram of a Websocket session, but at the same time, we can notice that the communication via Websocket protocol is a TCP-based one. This implies that before the client and the server can exchange data through Websocket protocol, it is mandatory to establish a TCP connection between the two sides. As it can be seen in the picture, there are needed three messages to be exchanged in order to start the connection (this phase is also called handshaking). For the purpose of having a Websocket-based communication, the two sides have to create a Websocket session because at this point, after the handshaking, there is possible only to transfer data through the plain TCP protocol.

To establish a session, client sends a Websocket Upgrade Request to the server, upon which server responds with a Websocket Upgrade Response [15]. The Websocket Upgrade Request is nothing more than an HTTP GET Request with the headers Connection and Upgrade telling the server the client's intention of using the Websocket protocol over the regular HTTP for the rest of the session. If there is possible to establish the Websocket session, the server sends back an HTTP response, with the status code having the value 101 Switching Protocols.

In paper [11], there was conducted a test to realize a comparison between Websocket and HTTP requests performances in asynchronous transmission. The results of the test are impressive, showing that the Websocket communication was ten times more efficient than the HTTP one. At the same time, other studies emphasize that Websocket's performance is much better than HTTP in terms of network traffic and delay, especially in large number concurrency case [11]. These results encourage us to use this technology in order to develop our application for the chat feature between doctors and patients.

4 Application Description

4.1 Introduction

This chapter focuses on our implementation of a patient-doctor relationship management application. We chose to implement it as a web application for the reasons presented in the previous chapter, such as the impressive evolution in this field in the last decades, but also for the ease of developing and maintaining a web software system.

As we have stated throughout this thesis, the application developed by us is only intended to improve the patient-doctor relationship and not completely replace face-to-face interaction. To achieve this, the first problem that arises is to identify the main features that the application should have.

Specific to any web application, each user should have an account with their personal information like first name, last name, email and a password, which he or she will use to log in to the application. Once the user has successfully accessed the application, its content should be displayed according to the type of user. A chat where doctors can communicate in real-time with their patients would be primarily for our topic. At the same time, the patients should have the possibility to make an appointment and the doctors must be able to see their schedule for a certain period. It would be nice for patients to write their feelings and observations about the disease's evolution in a special section and for doctors to have access to this information.

These are just the features that immediately came to mind when we set out to develop this application, but the truth is that there are many more than that. This is due to the fact that the problem we are trying to solve is a complex one, being extremely difficult to realize a perfect solution. That being said, in the following, we will present the features that we have implemented and consider the minimum necessary in an application to manage the patient-doctor relationship.

4.2 Features

As we have already mentioned, a feature that can be found in most web applications is the creation of accounts for each user. Thus, the first time a person wants to use our application, a login screen will be displayed, as can be seen in Figure 4-1. If the respective user has previously accessed the application and his or her session has not expired, then this screen will be replaced by the content of the application. We have set a session to be 24 hours, which means that if the user has not used the application for more than this period they must log in again.

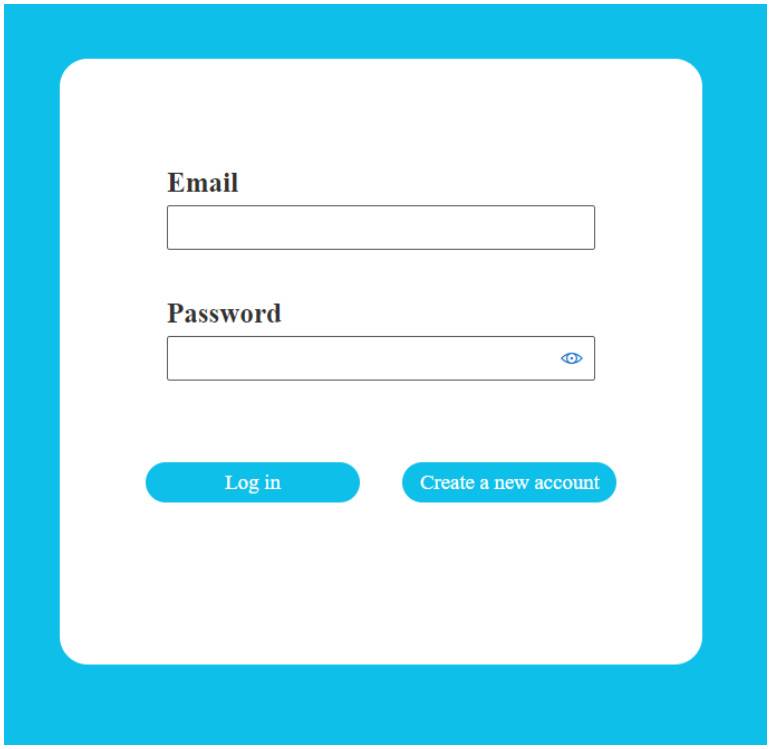

Figure 4-1 Login screen

If the user does not already have an account, then he or she can press the “Create a new account” button, which will send the user to the registration screen. As it can be seen in Figure 4-2, there are requested some general information like the name of the user, an email that must be unique and a password that has to be repeated. The most important thing to mention in this part is that only a patient account can be created. The reasons behind this decision are simple: it is unacceptable for an ordinary person to create an administrator account and also, it is not a wise

idea to let anyone pretend to be a doctor. For these arguments, only an administrator account is authorized to add a doctor to the application. The doctor should contact an administrator and prove that he or she has this profession; after this, the administrator will create a doctor account for that person.

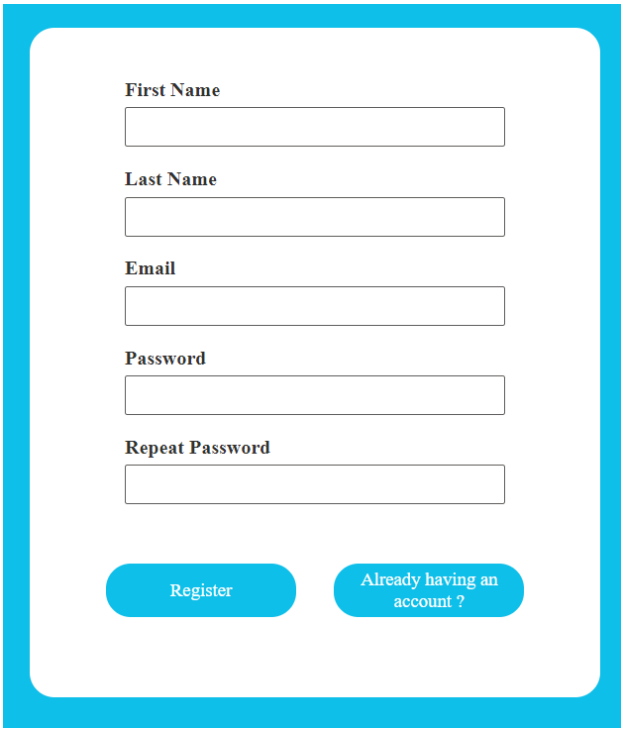

Figure 4-2 Register screen

After the administrator has successfully logged into the application and accessed the page specific to this type of user, the content displayed will be the one presented in Figure 4-3. The design is intuitive, being four categories of entities on which changes can be made: Accounts, Medicines, Diseases and Specializations. When selecting an option, under the menu with possible operations there will be displayed the content for the selected option. For example, if the administrator selects the option Update an existing disease and selects one disease from the dropdown list, then what will be displayed is described in Figure 4-4. Beside these, the selected option will have the same color, but the other options' color will change to gray. For the other options, the content that will be shown is similar. When the selected option is one to add an entity, the required fields will be empty, but for an update action, the administrator must firstly select from a dropdown an entity to be modified and then the fields will be initialized with the

information for that entity. This was done to give the administrator the possibility to change only certain fields and not all of them.

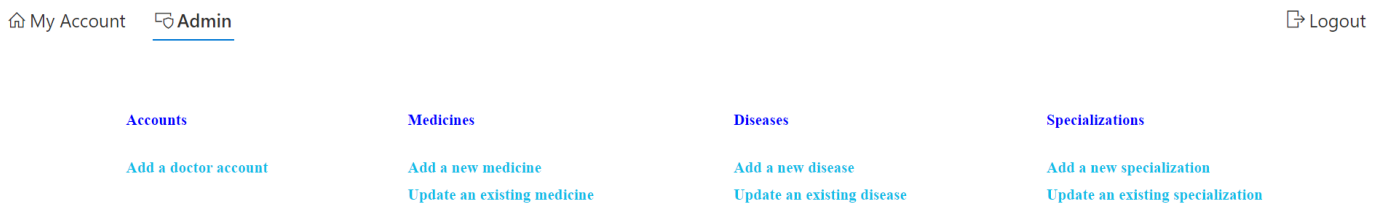

Figure 4-3 Administrator features

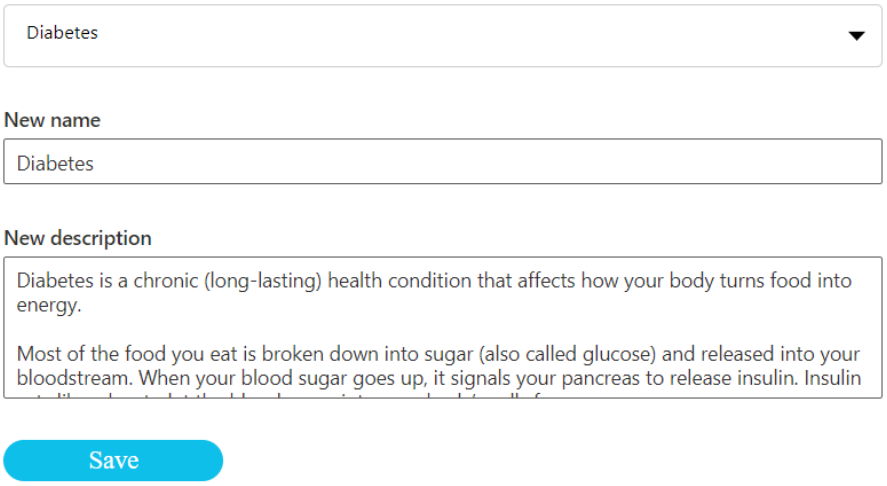

Figure 4-4 Update an existing disease feature

In the following, we will present the features of a doctor's account. Immediately after the doctor has successfully logged into the application, the content displayed will be as the one shown in Figure 4-5. At the top of the page we can notice a menu bar with the following options: My Account, See my appointments, Chat, Feedback, Recipes, Information and Logout. Initially, there is selected the My Account option. For this case, the information about the doctor is displayed together with the possibility to change the email or the password by clicking the "Modify" button. When this happens, the email field becomes enable and there are 2 more fields

for password and repeating of the password and 2 buttons: one for saving the updated information and one for cancelling this action.

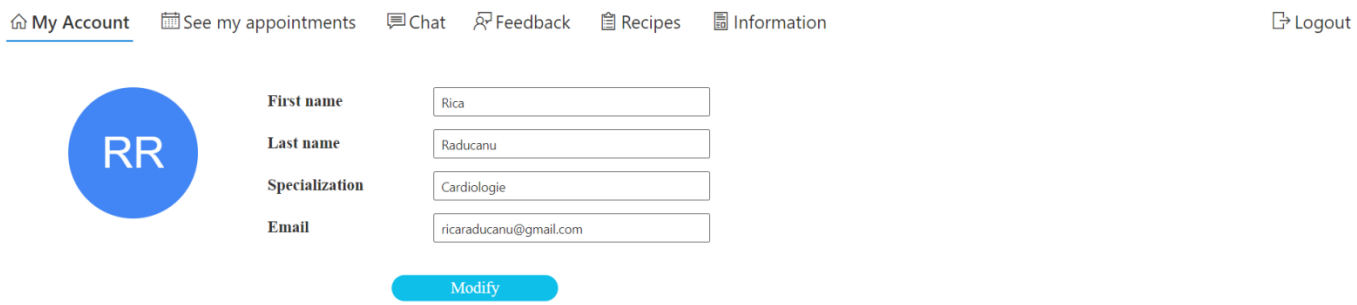

Figure 4-5 Doctor Perspective

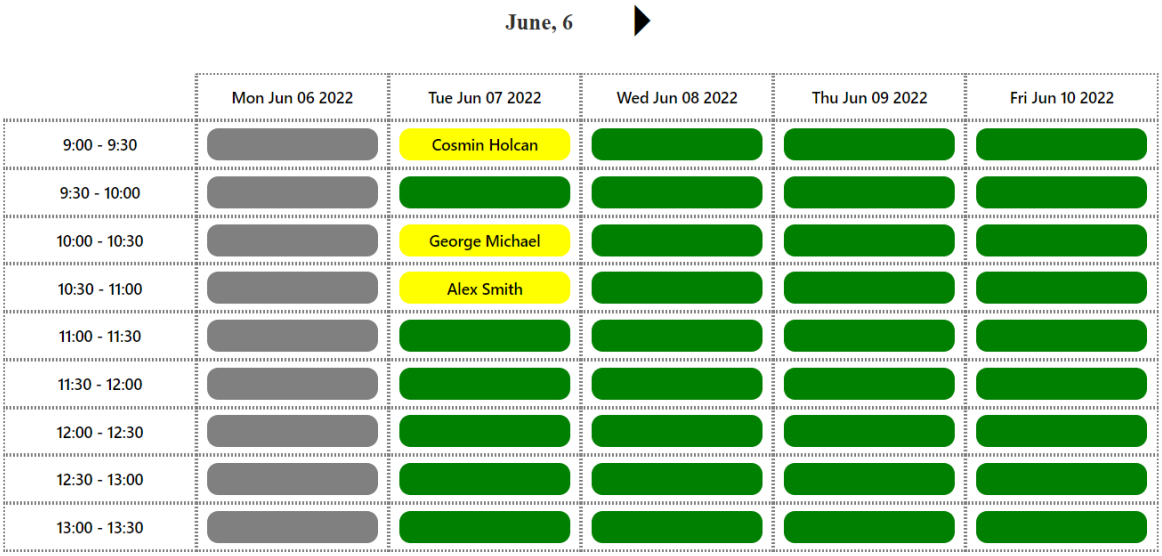

Figure 4-6 Calendar feature

By clicking the second option, See my appointments, the user will be redirected to the calendar page. On this page, the doctor can see his or her schedule over a week, like it is shown in Figure 4-6. On the top of the calendar will be the date of Monday of the current week and an arrow to right. By clicking this arrow the content is changed to show the data for the next work week. In this case, an arrow to left will be shown in order to navigate to the previous week. The structure of the calendar is intuitive, being intervals of 30 minutes for an appointment, from 9:00 to 17:00.

For the past dates, the color of the slot is gray. For the remaining slots, if there is an appointment for someone, the color is yellow and the name of the person is displayed, otherwise, the color of the slot is green.

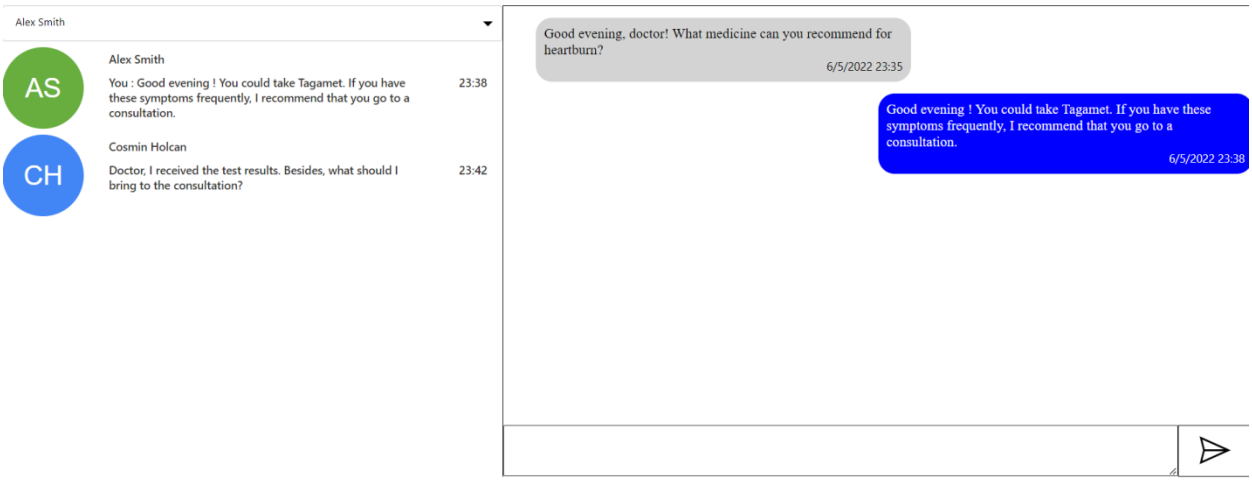

Figure 4-7 Chat page

In figure 4-7, we can see the chat page. In the top left corner is a dropdown with patients and under, it a list with the conversations of the current logged-in doctor. The doctor can choose either from the dropdown or from the list of conversations a patient. After this happens, in the right part the messages between them will be displayed. The messages from the-logged in user are on the right side and from the other person on the left side, having different colors. There is a box under the messages section to write a new message and a button in the right part to send it.

In the Feedback section, there are 2 dropdowns: one to choose a patient and one to choose a disease. After the doctor selects one option from each dropdown, on the right side are the observations of the doctor about the disease's evolution and on the left part are the notes of the patient.

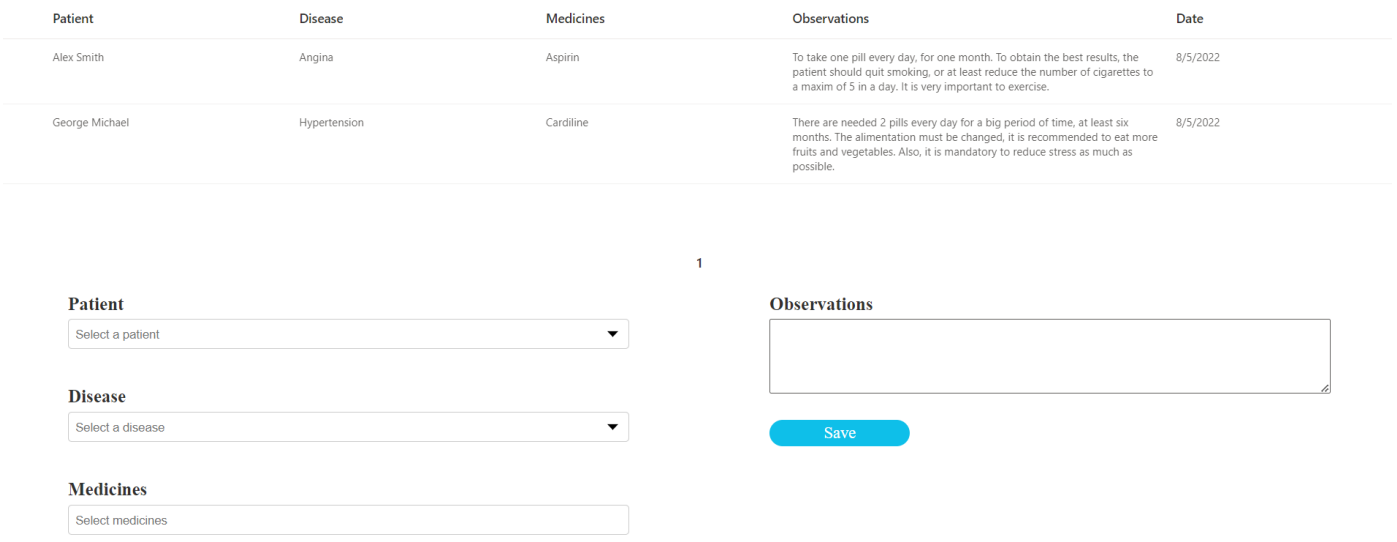

Figure 4-8 Recipes Page

The Recipes page contains a table with the recipes prescribed by the current doctor to different patients. The columns have intuitive names in order to make it easier for the user to understand the content. Thus, there is displayed the Patient for whom the recipe is, the Disease that he or she is suffering, a list with the prescribed Medicines, some Observations made by the doctor and the Date when the recipe was prescribed. Under the table, doctor can add a new medicine by selecting the required data: a patient, a disease and some medicines; the observations are optional.

The Information page is more like a library where patients and doctors can find information about a disease or medicine, being in fact the Description field introduced by the administrator. There is a dropdown list on the left side with all diseases from which users can select one and under the dropdown will be displayed information about that disease. The behavior for the medicines is exactly the same on the right side.

In the case that the current logged-in user is a patient, the content will suffer modifications. The first one is that the name of the calendar page will be changed to “Make an appointment” in the menu from the top of the page. The colors of the slots are changed too: if a slot is occupied, its color is red, if one is free then the color is blue and the user can make an appointment for that

interval by clicking the respective spot and if the slot was already booked by him the color is yellow. The chat page is almost identical with the difference of people from the dropdown list, being only doctors to select from. On the feedback page, the patient can't see what are the observations of the doctor, being displayed only what he or she wrote about the disease. The recipe page doesn't contain obviously the ability to prescribe a new recipe and the column Patient is replaced by Doctor. The Information page is exactly the same as for the doctor.

4.3 Implementation

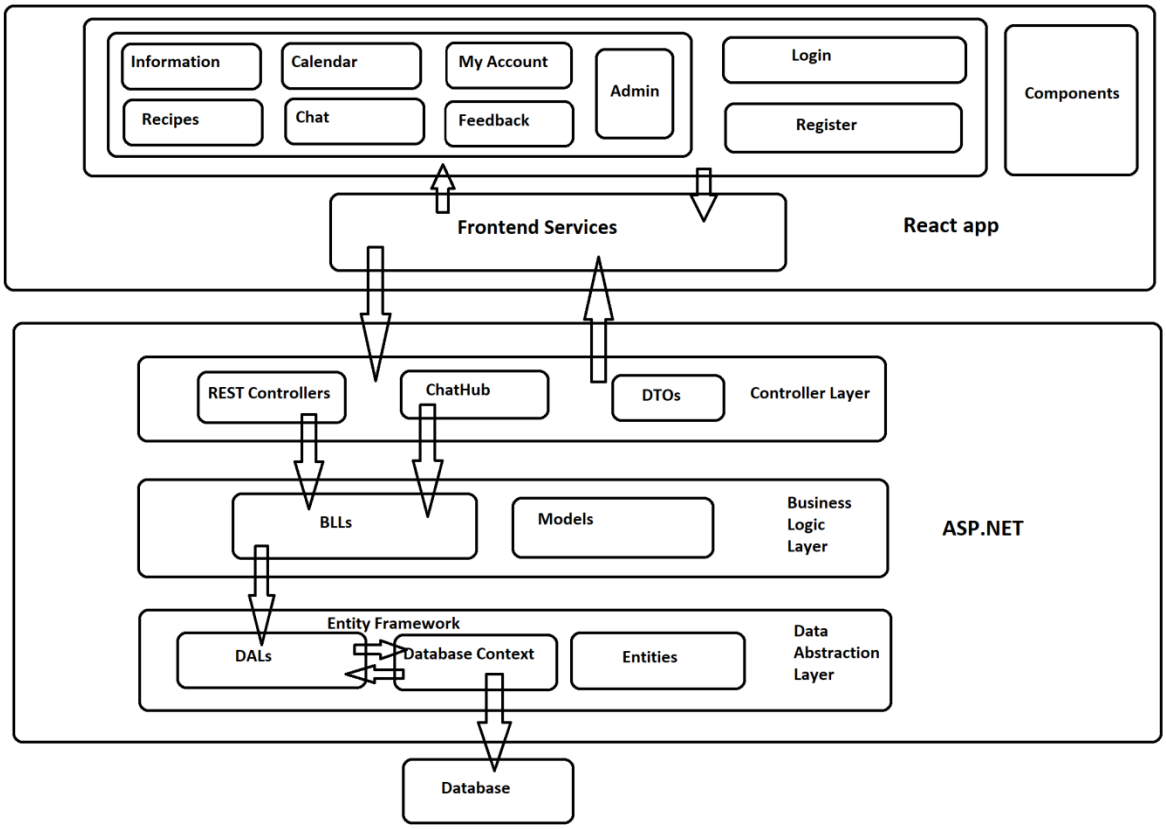

Figure 4-9 Application architecture

The application is divided in two parts: backend and frontend. A diagram with the architecture of the application can be seen in Figure 4-9. As we can see, the backend was realized with ASP.NET technology and the frontend with React. The backend side is composed of three layers: Controller Layer, Business Logic Layer and Data Abstraction Layer, each one having specific responsibilities that will be discussed further. The frontend side has a different structure, being one specific to React applications. Each page is in fact a functional component that can also use smaller components. In this way, there are created functional structures that can be easily reused. We will explain more about these things later, now focusing on the structure of the backend.

The **Data Abstraction Layer** is responsible for the communication with the database. It was implemented as a Class library. After its content is build, it creates a DLL (dynamic link library). In this layer, there are created the entities that we use in the application. Practically, we have classes for each entity of our project, like Doctor, Patient, Treatment, Disease or Medicine. Then, we have a class called PatientDoctorManagementDbContext that makes the connection with the database. In this class, there is a database set (DbSet) for each entity that corresponds with its table from the database. So, when we add a new entity, let's say a Specialization, we use the respective DbSet from the class PatientDoctorManagementDbContext, we save the changes performed on the DbSet and the table with specializations from the database is updated. We used a code-first approach when creating the entities and the database. With the Code-First approach, the developers can focus on the domain design and start creating classes as per your domain requirement rather than design your database first and then create the classes which match your database design [10]. Next, we have a BaseDAL class and for each entity a DAL class for that entity that inherits from BaseDAL and will perform operations on that entity's database set.

The BaseDAL class code is simple, containing only an object of type DALContext that has as member access modified the keyword protected. In this way, all DAL classes will access the same DALContext object. On the other hand, the code for DALContext is more complex as what we wanted to do was to use the concept of Lazy Initialization for the DAL classes and PatientDoctorManagementDbContext and to instantiate these DAL classes only once. The arguments for these decisions are the improving the performance and reducing the used memory. The most important aspects of the DALContext class implementation can be seen in Figure 4-10.

```
#region Constructors
public DALContext()
{
    _dbContext = new Lazy<PatientDoctorManagementDbContext>(() => new PatientDoctorManagementDbContext());
    _administratorsDAL = new Lazy<AdministratorsDAL>(() => new AdministratorsDAL(this));
    _appointmentsDAL = new Lazy<AppointmentsDAL>(() => new AppointmentsDAL(this));
    _diseasesDAL = new Lazy<DiseasesDAL>(() => new DiseasesDAL(this));
    _doctorsDAL = new Lazy<DoctorsDAL>(() => new DoctorsDAL(this));
    _feedbacksDAL = new Lazy<FeedbacksDAL>(() => new FeedbacksDAL(this));
    _medicinesDAL = new Lazy<MedicinesDAL>(() => new MedicinesDAL(this));
    _messagesDAL = new Lazy<MessagesDAL>(() => new MessagesDAL(this));
    _patientsDAL = new Lazy<PatientsDAL>(() => new PatientsDAL(this));
    _specializationsDAL = new Lazy<SpecializationsDAL>(() => new SpecializationsDAL(this));
    _treatmentsDAL = new Lazy<TreatmentsDAL>(() => new TreatmentsDAL(this));
}
#endregion

#region IDisposable
public void Dispose()
{
    Dispose(true);
    GC.SuppressFinalize(this);
}

public virtual void Dispose(bool disposing)
{
    if (disposing)
    {
        if (_dbContext.IsValueCreated)
            _dbContext = null;

        if (_administratorsDAL.IsValueCreated)
            _administratorsDAL = null;

        if (_appointmentsDAL.IsValueCreated)
            _appointmentsDAL = null;
```

```
~DALContext()
      {
          Dispose(false);
      }
      #endregion
```

Figure 4-10 DALContext class implementation

The **Business Logic Layer** was also implemented as a class library. We had to connect this layer to the previously discussed one by adding a reference to the DLL generated by Data Abstraction Layer. In this way we establish a one-way connection between these two layers as we intended; the Data Abstraction Layer should not be able to use code from the Business Logic Layer. The purpose of this layer is to execute the specific business rules of the application, being in fact an intermediary layer between the Controllers Layer and the Data Abstraction Layer. Its structure is similar to the one of the Data Abstraction Layer, the difference, in this case, being that the BLLContext class uses Lazy Initialization for an object of type DLLContext. Thus, all BLL classes will be able to use the DLLContext and also its members. In the Business Logic Layer are also present the Models for our project, being classes inspired by the entities but with a change structure (fewer fields, different names, etc.) that will be used for the frontend part.

The **Controller Layer** deals with the processing of requests from the front end side and contains also the class ChatHub for real-time messaging. For each entity, we have a separate controller class and also an AuthorizationController class. It is time to say that good practice in terms of security is to encrypt the passwords and not save them in plain text in the database and we occupied with this aspect. Also, when receiving a request that includes operations for an already logged-in user we want to be sure the user is really logged into the application. So, for each user, we create a JWT Token when he or she logged in and after this, the token will be used whenever the user makes a request. Also, it is verified the type of user in case of some operations (we refer especially to administrator functionalities). For the ChatHub class, we used an ASP.NET library called SignalR that simplified our work, being implemented with the help of Web sockets.

In the following, we will discuss about the implementation of the **Frontend** side. In the beginning, there was implemented a Router to redirect the user to the login page if he or she is not logged in or the session has expired. To determine this, we take the JWT token from the Local Storage and try to refresh this token. If there is no token on Local Storage or the refresh operation receives a BadRequest response from the server, then the Login page will be displayed. Also, in these cases, the user cannot access the content of the application until he or she logs in successfully. After this happens, the token will be modified automatically at intervals of 30 minutes. At the same time, when the user is logged in there is made a connection of type Web socket with the backend. As for the implementation of the pages, all that must be said is that firstly there were created smaller function components that were used in multiple places, this being a common practice for the React applications. In order to have a better structure of the whole project, each component has three files: one for its styling having the suffix .types.ts, one containing the definitions of its used types having the suffix .types.ts and one for the implementation with the extension .tsx. An example is given by the CustomCallendar component and it can be seen in Figure 4-11.

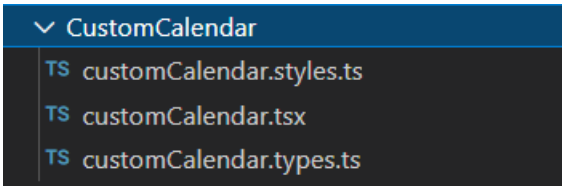

Figure 4-11 CustomCallendar component structure

In figure 4-12 can be seen the structure of the database. As we said, there was used a code-first approach meaning that the database was automatically generated based on our entities from the Data Abstraction Layer.

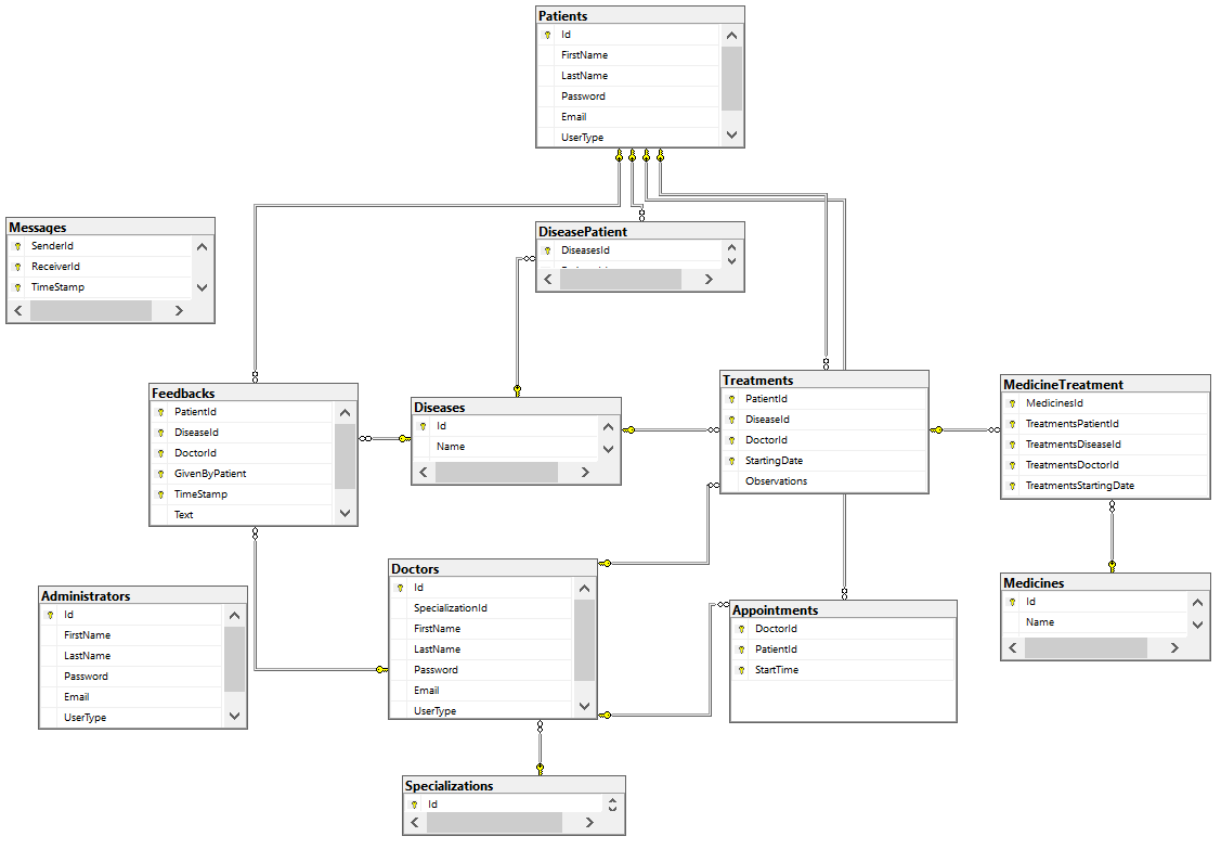

Figure 4-12 Database structure

4.4 Used technologies

In the development process of the application, there were used different technologies like programming languages or frameworks and in the following, they will be presented.

The backend was implemented using **ASP.NET Core** facilities. According to Microsoft Corporation ASP.NET is unified Web development model that includes services necessary to

build enterprise web applications with a minimum of coding and offered on a free licensing agreement. It is event-driven programming model and part of the .NET Framework [14]. ASP.NET is typically run on IIS (Internet Information Services), which is not supported on GNU/Linux by Microsoft; thus, Kestrel must be used instead [9].

ASP.NET Core is the open-source version of ASP.NET. It is a rewrite, which is meant to provide a slimmer, but more up-to-date set of features, as well as a new lightweight web server, Kestrel (although using IIS is still possible on Windows) [9]. Most often, ASP.NET Core applications use the **C#** language as we did in developing the application. C# is a modern, type-safe, object-oriented programming language. For making the connection with the database and working with the data from the database in form of domain objects we used **Entity Framework**. It represents an Object Relational Mapping developed for .NET applications, enabling the programmers to work at a higher level of abstraction by removing the need of writing code directly on the tables from the database.

Regarding the frontend side, we used **React.JS** which is a free and open-source library written in JavaScript. React Hooks represents a feature of the framework that makes it more and more popular among developers. They allow function components to access the state without writing a class. We also used two hooks in our application: useState and useEffect. The first one lets us modify the content of a page when an object changes (for example, when a user on the Information page selects a different disease then the description text will be automatically changed). The second hook is used to execute a function whenever an object from a list of dependencies is modified. It must be said that React only deals with the management of the state and its rendering, so in many cases, there must be used other components from multiple libraries.

5 Conclusions

The problem studied by us in this thesis, the patient-doctor relationship implemented in a software system, will most likely remain relevant for a long time. This is due to the continuous digitization process in which society is. Our goal was to answer a series of questions, such as what are the types of patient-doctor interaction, how do they influence the development of the application, and what is the structure of a web application? Regarding the first questions, we found answers by analyzing multiple research papers.

As we already explained, there are four main patient-doctor relationship models: the paternalistic model, the informative model, the interpretive model and the deliberative model. In the last decades, there was a shift from a paternalistic approach to a deliberative one, in which the decisions are made jointly by the doctor and the patient. One presented methodology for making a software system for our problem is SEMHTA. Of course, in the development process of our application, we could not follow all the recommendations of this methodology, as it would have been necessary to involve the medical staff and more programmers.

There are several structures for a web application, but the most common one is the three-tier one. In order to achieve real-time communication, the most effective solution is to use web sockets as they have better performance results in this case when compared to classic HTTP Requests.

The application which we developed represents just a way in which the problem of creating a software system for the patient-doctor relationship can be solved. It is far from a perfect application, as the proposed problem is a complex one that requires more resources for an ideal implementation. Further development implies more functionalities like setting a profile picture to identify easily a user or the creation of completely virtual meetings. In order to make the best possible product there can be considered ideas like keeping patients' medical information in a more secure way, such as using blockchain technology or using artificial intelligence to automatically generate treatments.

Bibliography

[1] Agarwal AK, Murinson BB, *New dimensions in Patient-Physician Interaction: Values, Autonomy, and Medical Information in the Patient-Centered Clinical Encounter*, Rambam Maimonides Med. J., 2012-07-31, 3(3): e0017

[2] A. Al-Dahmash, S. El-Masri, *A New Proposed Software Engineering Methodology for Healthcare Applications Development*, Int J Mod Eng Res. 2013;3(3):1566-70

[3] Balint M, *The Doctor, His Patient, and the Illness*, The Lancet, 1955-04-02, Volume 265, Pages 683-688

[4] Borza L, Gavrilovici C, Stockman R, *Ethical models of physician-patient relationship revisited with regard to patient autonomy, values and patient education*, Rev. Med. Chir. Soc. Med. Nat., Iasi, 2015, Volume 119, No 2, Pages 496-501

[5] G. Candea, A. Fox, *Designing for High Availability and Measurability*, 1st Workshop on Evaluating and Architecting System Dependability (EASY), 2001

[6] Emanuel E, Emanuel L, *Four Models of the Physician-Patient Relationship*, JAMA: the Journal of the American Medical Association, 1992-09-16, Volume 267, Pages 2221-2226

[7] P. E. Idoga, M. Agoyi, E. Y. Coker-Farrell and O. L. Ekeoma, *Review of security issues in e-Healthcare and solutions*, 2016 HONET-ICT, 2016, pp. 118-121

[8] Kaba R, Sooriakumaran P, *The evolution of the doctor-patient relationship*, International Journal of Surgery, 2007-02-01, Volume 5, Pages 57-65.

[9] Kronis, Kristiāns & Uhanova, Marina, *Performance Comparison of Java EE and ASP.NET Core Technologies for Web API Development*, Applied Computer Systems, May 2018, vol. 23, no. 1, pp. 37-44

[10] Lerman J, Miller R, *Programming entity framework: code first*, 2011, O'Reilly Media, Inc.

[11] Liu, Q.; Sun, X., *Research of Web Real-Time Communication Based on Web Socket*, Int. J. Commun. Netw. Syst. Sci.2012,5,797–801.

[12] Marchetto A, Tonella P, Ricca F, *"State-based testing of ajax web applications" in 1st Int. Conf. on SoftwareTesting, Verification, and Validation*, Lillehammer, Norway, pp. 121–130, 2008

[13] McGuire, A., Roberts, J., Aas, S., & Evans, B., *Who Owns the Data in a Medical Information Commons?*, Journal of Law, Medicine & Ethics, 2019, 47(1), pp. 62-69

[14] Mishra, Atul, *Critical Comparison Of PHP And ASP.NET For Web Development*, International Journal of Scientific & Technology Research, July 2014, vol. 3, issue 7

[15] D. Skvorc, M. Horvat and S. Srbljic, *Performance evaluation of Websocket protocol for implementation of full-duplex web streams*, 2014 37th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), 2014, pp. 1003-1008

[16] Vadlamudi S G, Sengupta S, Taguinod M, et al., *Moving target defense for web applications using Bayesian stackelberg games*, in Proc. of the 15th International Conference on Autonomous Agents and Multiagent Systems, 1377-1378, 2016